TASK 5 – Read Input and Convert Types

Input and conversion

input() always returns a str, even when the user types digits. Convert identifiers and whole counts with int() only when appropriate; convert litres, fat and SNF to Decimal from their text for financial-quality calculations. float("8,5") raises ValueError because Python expects a decimal point; float("eight") also raises ValueError. Surrounding whitespace is accepted by float after conversion, so float(" 8.5 ") works.

Validation approaches

String methods such as isdigit() are useful for unsigned whole numbers but are not sufficient for decimal values, signs or scientific notation. A try/except ValueError around Decimal(text) or float(text) is clearer and handles malformed input safely. The program should also reject NaN and infinity, zero or negative litres, and out-of-range quality readings. Do not silently replace a comma with a decimal point unless the co-operative explicitly adopts that locale rule.

Example input routine and aligned output

```
from decimal import Decimal, InvalidOperation

def read_decimal(prompt):
    while True:
        text = input(prompt).strip()
        try:
            value = Decimal(text)
            if not value.is_finite():
                raise ValueError("Enter a finite number")
            return value
        except (InvalidOperation, ValueError):
            print("Invalid number. Example: 8.5")

amount = Decimal("283.90")
print(f"Amount payable: Rs {amount:8.2f}")
```

Alternative and complexity

Pre-checking with string methods takes O(m) time for an input string of length m and can be difficult to make complete. try/except conversion also takes O(m) time and delegates numeric syntax validation to Python. It accepts surrounding spaces, rejects "8,5" and rejects "eight". For this reason, direct conversion with exception handling is recommended. Each entry is processed in O(1) with respect to the number of farmers (or O(m) in the number of input characters), and only a fixed number of temporary values is required.